

Anonymous submission

Abstract

Long-horizon agents must use past interaction to guide future decisions, but past interaction cannot be placed in context verbatim at every turn. Prior work has largely studied how experience is externalized and compressed into reusable artifacts such as episodic memories, state summaries, procedural skills, declarative rules, traces, and workspace files. This paper studies the complementary runtime problem: how a current intent accesses such typed artifacts and exposes them to a downstream model in a form that is sufficient for decision making. We introduce Evidence View Realization, a framework that separates compressed artifact storage from typed access primitives, primitive traces under budget, evidence-view construction, and downstream decoding. The framework predicts that memory failures need not be retrieval failures: a system may store or even retrieve the right artifacts while failing to expose the roles, bindings, temporal authority, aggregation structure, or conflict status needed by the current decision. We operationalize this view through planned diagnostics on long-term personalized memory QA and stale-state validity benchmarks, using controlled evidence settings to separate access, selection, view construction, and decoding failures. The final empirical contributions and quantitative claims will be filled after experiments are complete.

1 Introduction

Large language model (LLM) agents are moving from single-session assistants to persistent systems that act across long horizons, multiple tasks, and evolving user states. In such settings, future decisions depend on past interaction: user preferences, task commitments, tool outcomes, failed attempts, environmental changes, and previously learned procedures. Scaling the context window is an incomplete solution. Even when longer histories can be provided, relevant information may be buried, outdated, displaced by irrelevant context, or present but unused. Long-horizon agents therefore need mechanisms that make past interaction usable at decision time.

Recent work has made substantial progress on this problem by externalizing interaction history into reusable substrates: user states, episodic records, relational stores, graphs,

semantic workspaces, skills, rules, and compressed experience artifacts. Experience-compression frameworks make this trend explicit: raw interaction traces can be compressed into artifacts at different abstraction levels, trading specificity for reusability and lower context cost (Zhang et al. 2026a). Runtime access has also moved beyond flat top- k retrieval. Modern systems combine retrieval with reranking, graph traversal, hierarchical selection, source reading, tool execution, workspace navigation, and trace-based attribution. These mechanisms improve the geometry of access: they make relevant artifacts easier to locate, select, and expose.

These developments sharpen the access layer, but they also expose its limit. The field has become good at making evidence easier to find. It has not solved what happens after evidence is found. In long-term personalized memory QA, aggregation-heavy questions can fail under oracle-evidence conditions, where the correct evidence is already present. This is not a retrieval failure. It is a post-access failure: existing systems optimize the geometry of access, while hard memory use also requires the algebra of evaluation.

We study this post-access regime. Access determines which evidence reaches the model; post-access evaluation determines whether that evidence is exposed in a decision-sufficient view, whether variables are stably bound across evidence items, and whether the final answer is constrained to the evidence-defined valid set. A flattened textual view may present a timestamp and a caption while omitting the relations, quantities, source pointers, or modality-specific structure needed for aggregation. The evidence is in context, but the view is not decision-sufficient.

This distinction matters because language-only decoding is a weak evaluator over recalled evidence. Without explicit variable binding and constraint enforcement, the model can match, paraphrase, and classify fluently, but it does not reliably execute multi-evidence aggregation, conflict resolution, or discrete computation. Soft controls such as instruction tuning, prompting, chain-of-thought, and schema formatting can bias the model toward aggregation-like trajectories, but they do not enforce the answer set.

We use Artifact Access Semantics to organize the runtime path from compressed artifacts to access operations, primitive traces, evidence views, and decisions. Within this path, our focus is the post-access boundary where recalled evidence must become an answerable state. We formalize the

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

access–evaluation separation and instantiate it in long-term personalized memory QA under controlled evidence conditions. Our thesis is simple: multimodal long-term memory systems need both evidence access and evidence aggregation. Existing systems have advanced the first substantially; the second—view construction, variable binding, operator execution, and evidence-valid decoding—remains the bottleneck.

2 Artifact Access Semantics

The *Experience Compression Spectrum* (ECS) studies how an agent’s interaction trace is transformed into reusable artifacts (Zhang et al. 2026a). Following ECS, let an interaction trace be

$$T = \{(s_t, a_t, o_t, f_t)\}_{t=1}^N, \quad (1)$$

and let a level- L compression function be

$$C_L : T \rightarrow K_L. \quad (2)$$

Different choices of L produce artifacts at different abstraction levels, such as raw traces, episodic memories, procedural skills, declarative rules, state summaries, or workspaces. This axis answers a storage-side question: what does past interaction become?

We study the complementary runtime question: once past interaction has been compressed into artifacts, how does a current intent use them for a future decision? A compressed artifact is not only a content payload. It also carries usage semantics: a trace can be searched or replayed, a state can be queried, a graph can be traversed, a skill can be invoked, a rule can be injected or checked, and a workspace can be read, edited, or verified. We call this runtime problem *Artifact Access Semantics* (AAS). Its core chain is

$$\begin{aligned} \text{compression} &\rightarrow \text{artifact} \rightarrow \text{usage semantics} \\ &\rightarrow \text{access} \rightarrow \text{decision}. \end{aligned} \quad (3)$$

2.1 From Access Interfaces to Decision Views

AAS treats retrieval as one access interface rather than the general form of memory use. Let $\mathcal{P}$ be the set of typed access primitives exposed by a memory system. At runtime, a current intent u induces an access trace

$$\tau_\pi(u, \mathcal{A}) = ((\rho_1, \theta_1, r_1), \dots, (\rho_T, \theta_T, r_T)), \quad (4)$$

where ρ_t is an access primitive, θ_t its parameters, and r_t its returned result. The trace may be a one-shot retrieval call, or a multi-step process involving graph traversal, source reading, workspace interaction, execution, or verification. The returned observations are then organized into a decision-facing view $v = \mathcal{V}(u, \tau_\pi)$ and passed to a downstream model.

This abstraction separates three questions that are often collapsed in end-to-end memory QA: whether the relevant artifact exists, whether a budget-feasible access trace exposes it, and whether the exposed view is sufficient for the current decision. Table 1 sketches how common systems occupy this space.

2.2 Diagnostic Scope

The rest of the paper focuses on the boundary after relevant artifacts, or oracle evidence, are available. This is intentionally narrower than the full memory lifecycle. We do not attempt to solve how memories are written, updated, indexed, or retrieved in general. Instead, we ask whether available evidence is exposed in a decision-facing form and whether the decoder can use it.

We distinguish two downstream failures. A *view-use failure* occurs when needed evidence exists and can be accessed, but the exposed view omits decision-critical structure such as bindings, temporal authority, source relations, aggregation fields, modality-specific information, freshness, or conflict status. A *decoder-use failure* occurs when the view exposes the required information, but the decoder still fails to produce an evidence-valid answer. This diagnostic scope motivates the theory in Section 3: access can make evidence reachable, but post-access evaluation must make it answerable.

3 Post-Recall Evidence Evaluation

We study memory-based question answering after the relevant evidence has been recalled. The point is not that retrieval is unimportant: retrieval determines whether the needed evidence is available. Our focus is the next stage. Once evidence is available, the system must expose a decision-sufficient view, bind the relevant variables, execute the required operations, and produce an answer whose denotation is valid under the evidence. We refer to retrieval-side mechanisms as *access geometry* and post-recall reasoning mechanisms as *evaluation algebra*.

3.1 Evidence-Defined Validity

Let memory be a finite typed evidence structure $\mathcal{A} = (E, R_1, \dots, R_m)$, where E is a set of evidence items and the relations R_j range over items, entities, timestamps, sources, amounts, statuses, locations, events, and states. Relations may include attributes such as $\text{amount}(e, a)$, $\text{status}(e, s)$, and $\text{time}(e, t)$, as well as structural edges such as $\text{newer-than}(e_i, e_j)$, $\text{overrides}(e_i, e_j)$, $\text{same-event}(e_i, e_j)$, and $\text{contradicts}(e_i, e_j)$.

A query q induces a semantic answer relation φ_q . The evidence-defined answer set is

$$\mathcal{Z}_q(\mathcal{A}) = \{z \in \mathcal{Z} : \mathcal{A} \models \varphi_q(z)\}, \quad (5)$$

where $\mathcal{Z}$ may contain entities, numbers, records, lists, state decisions, claim sets, or structured tuples. A natural-language response $y \in \Sigma^*$ is evaluated through a partial semantic projection $\eta : \Sigma^* \rightarrow \mathcal{Z}$. The answer is evidence-valid iff

$$\eta(y) \in \mathcal{Z}_q(\mathcal{A}). \quad (6)$$

This target is independent of surface fluency: the model is correct only when its output denotes an element of the evidence-defined answer set.

Let an access module return a query-conditioned substructure $\mathcal{A}_q = R_{\text{access}}(q, \mathcal{A}) \preceq \mathcal{A}$. Let $\mathcal{A}_q^*$ denote a sufficient evidence substructure for answering q . We call access oracle

Line of work	Artifact substrate	Runtime access interface	Exposed decision view
Long-context agents	Raw interaction history	Full-context reading	Long prompt context; history is present but not necessarily localized or updated.
RAG / memory retrieval	Text chunks or memory records	Top- k retrieval and reranking	Ranked evidence passages.
GraphRAG / hierarchical memory	Entity, relation, or category graphs	Graph traversal, neighborhood expansion, or global selection	Graph-selected evidence; structure guides access.
Episodic / workspace memory	Actor-state-event workspaces	Entity, event, or state lookup	Structured episodic summaries or state-like views.
Memory operating systems	Persistent user or task states	State lookup, update, and personalization hooks	Personalized memory context.
Direct corpus / code agents	Raw corpora, files, repositories	Grep, read, shell, execution, navigation	Composed evidence or verified result.

Table 1: AAS view of runtime artifact access. The map distinguishes artifact substrates, access interfaces, and the decision views exposed downstream.

when $\mathcal{A}_q^* \preceq \mathcal{A}_q$. Post-recall evaluation studies the conditional regime in which oracle evidence is present. Successful answering then requires

$$\text{Success} = \text{AccessOK} \wedge \text{ViewOK} \wedge \text{EvalOK}. \quad (7)$$

Our experiments condition on AccessOK and measure the remaining failures due to view insufficiency and evaluation error.

3.2 Decision-Sufficient Views

The recalled evidence is not consumed by the model as $\mathcal{A}_q$. It is exposed through a view and a serialization,

$$\mathcal{A}_q \xrightarrow{\text{View}_q} S_q \xrightarrow{\text{Ser}_q} V_q. \quad (8)$$

The view decides which fields, timestamps, statuses, source identifiers, modality markers, and relations are exposed. The serialization decides whether these elements appear as flat text, source-indexed records, a table, explicit relation triples, state slots, or a tool trace.

A view can include all retrieved evidence and still be insufficient for the decision.

Definition 1 (Decision-sufficient view). A view V_q is decision-sufficient for query q if, for any two evidence structures $\mathcal{A}_1, \mathcal{A}_2$,

$$V_q(\mathcal{A}_1) = V_q(\mathcal{A}_2) \implies \mathcal{Z}_q(\mathcal{A}_1) = \mathcal{Z}_q(\mathcal{A}_2). \quad (9)$$

The contrapositive is the useful part. If

$$V_q(\mathcal{A}_1) = V_q(\mathcal{A}_2), \quad \mathcal{Z}_q(\mathcal{A}_1) \cap \mathcal{Z}_q(\mathcal{A}_2) = \emptyset, \quad (10)$$

then no decoder observing only V_q can be correct on both worlds. The decoder receives the same input in two worlds that require disjoint semantic answers. Thus, answer-relevant distinctions lost by the view are unrecoverable from the view alone.

Proposition 1 (View collapse impossibility). If Equation 10 holds, no deterministic or randomized decoder that observes only V_q can be evidence-valid on both $\mathcal{A}_1$ and $\mathcal{A}_2$ with probability one. The proof is in Appendix A.

This is the first nontrivial bottleneck after recall. The problem is not whether the evidence exists somewhere in

memory, but whether the exposed view preserves the variables and relations needed by the query. A flat text view may be decision-sufficient for a single-field lookup, but insufficient for temporal override, source-priority resolution, contradiction handling, or abstention under missingness. We therefore treat view sufficiency as a query-view property, not as a global property of a format.

Operationally, our benchmark pairs each query with a gold evaluation program. The program specifies the fields, relations, and state variables required for the answer. This lets us estimate whether a view is decision-sufficient by checking whether it exposes the required variables in a usable form.

3.3 Dynamic Propagation of Binding Error

Post-recall evaluation often requires binding multiple variables before applying an operator: a receipt to an amount, a booking to a date range, an old record to an update, a claim to its source, or a conflicting item to a priority rule. Attention provides soft alignment, not discrete binding. At evaluation step t , let the model read a soft-bound value with attention weights $\alpha_{ti} \geq 0$ and $\sum_i \alpha_{ti} = 1$:

$$\tilde{v}_t = \sum_i \alpha_{ti} v_i. \quad (11)$$

Let S_t be the correct support set for the variable needed at step t , suppose all values in S_t share the target value v_t^* , and define the binding mass

$$B_t = \sum_{i \in S_t} \alpha_{ti}, \quad D_t = \max_i \|v_i - v_t^*\|_2. \quad (12)$$

The important question is not the static perturbation alone, but how it propagates through a multi-step evaluation trajectory. Let

$$h_{t+1} = T_t(h_t, \tilde{v}_t), \quad h_{t+1}^* = T_t(h_t^*, v_t^*), \quad (13)$$

where h_t is the actual decision state and h_t^* is the ideal state under correct binding. Define $e_t = \|h_t - h_t^*\|_2$. Assume the local update is Lipschitz in the current state and the bound variable:

$$\|T_t(h, v) - T_t(h', v')\|_2 \leq \rho_t \|h - h'\|_2 + \beta_t \|v - v'\|_2. \quad (14)$$

Proposition 2 (Dynamic propagation of binding error).
Under the above conditions,

$$e_{t+1} \leq \rho_t e_t + \beta_t(1 - B_t)D_t. \quad (15)$$

Consequently,

$$e_T \leq \left(\prod_{t=0}^{T-1} \rho_t \right) e_0 + \sum_{s=0}^{T-1} \left(\prod_{u=s+1}^{T-1} \rho_u \right) \beta_s(1 - B_s)D_s. \quad (16)$$

The proof is in Appendix A.

This bound is the main theoretical prediction. Post-recall errors should not merely depend on whether evidence was recalled. They should depend on the length and stability of the evaluation trajectory. When $\rho_t < 1$, binding errors are damped; when $\rho_t \approx 1$, they accumulate; when $\rho_t > 1$, early binding errors may be amplified. Therefore, under oracle evidence, error should increase with binding count, operator count, override depth, conflict degree, and missingness decisions. This motivates the complexity sweeps in Section 4.

The same bound also explains why structured views and harnesses can help. A structured view can increase B_t by making the correct support easier to identify. A decision-state view can reduce T by exposing the operands needed for evaluation. An operator harness can remove unstable computation from the model trajectory and execute it externally.

3.4 Decision State and Harnesses Evaluation

A decision state is the set of typed variables needed by the gold evaluation program. For lookup queries, this may be one field. For temporal, conflict, or aggregation queries, it may include candidate records, stale records, timestamps, source priorities, override edges, conflict flags, missingness indicators, and numeric operands. The purpose of a decision state is not to preserve all recalled text, but to preserve the variables needed for the answer.

Language models produce proposal distributions over strings, $p_\theta(y \mid q, V_q)$. Evidence semantics defines which denotations are valid, $\eta(y) \in \mathcal{Z}_q(\mathcal{A})$. Prompting, chain-of-thought, and scratchpads reshape the proposal distribution. They may improve the internal trajectory, but they do not enforce evidence validity.

Harnessed evaluation changes the interface. Operator harnesses execute discrete operations such as sum, count, latest, override, join, and status comparison. Verifier harnesses reject candidates that violate source, type, timestamp, relation, support, or missingness constraints. The verifier does not merely prefer correct answers; it deletes invalid ones. When no proposal survives, the system should abstain, not from uncertainty, but from evidence-level non-authorization.

This distinction guides the empirical design. Prompting-only methods are tested as proposal-shaping baselines. Operator and verifier harnesses are tested as evaluation-centric interventions that either externalize unstable operations or restrict admissible outputs.

4 Experiments

The experiments test whether post-recall failures are structured, predictable, and reducible. We do not use the fact that aggregation is harder than lookup as a contribution. Lookup queries serve as a sanity check: under oracle evidence, they should be easy. The main evidence comes from three questions: whether evaluation complexity predicts failure, whether view insufficiency predicts error, and whether harnessed evaluation reduces semantic invalidity beyond prompting-only methods.

4.1 ATM-Bench Diagnostic Annotation

We use ATM-Bench as the base testbed because it evaluates long-term personalized memory QA over multimodal personal evidence, including images, videos, emails, metadata, and structured memory fields (Mei et al. 2026). To make post-recall failures measurable, we augment each analyzed instance with diagnostic annotations:

$$(\mathcal{A}, q, \mathcal{A}_q^*, P_q, \mathcal{Z}_q(\mathcal{A}), \mathcal{R}_q, \mathcal{L}_q). \quad (17)$$

Here $\mathcal{A}$ is the evidence structure, q is the query, $\mathcal{A}_q^*$ is the sufficient evidence substructure, P_q is the gold evaluation program, $\mathcal{Z}_q(\mathcal{A})$ is the semantic answer set, $\mathcal{R}_q$ is the set of required variables and relations, and $\mathcal{L}_q$ contains error labels for analysis.

The gold program P_q specifies the operators needed to answer the query. Operators include field selection, source selection, relation binding, joining, latest-record selection, override resolution, counting, summing, grouping, contradiction checking, and abstention under missingness. The program is used for evaluation and diagnosis; it is not necessarily provided to the model.

The required-variable set $\mathcal{R}_q$ specifies the fields, timestamps, statuses, source identifiers, relation edges, operands, and state indicators required by P_q . This enables view-level analysis. A view is marked insufficient for q if it fails to expose elements of $\mathcal{R}_q$ in a usable form. We define the empirical view insufficiency rate of a view family V as

$$\text{VIR}(V) = \Pr_{q, \mathcal{A}} [V_q \text{ omits a required variable or relation from } \mathcal{R}_q]. \quad (18)$$

This avoids treating flat text, tables, or state views as globally sufficient or insufficient. Sufficiency is measured at the query-view level.

4.2 Main Hypotheses

H1: Evaluation complexity predicts post-recall failure.

Under oracle evidence, semantic error should increase with binding count, operator count, override depth, conflict degree, and missingness level. This tests the dynamic propagation prediction from Section 3.3.

H2: View insufficiency predicts error. Query-view pairs with higher view insufficiency should have higher post-recall error. Structured views and state views should reduce errors when they expose the variables and relations required by the gold program.

H3: Harnessed evaluation reduces semantic invalidity. Operator and verifier harnesses should reduce evidence-invalid answers more reliably than direct answering, chain-of-thought, or scratchpad prompting, especially for aggregation, temporal, conflict, and abstention queries.

The lookup-versus-aggregation comparison is reported only as a diagnostic. It verifies that the oracle-evidence protocol is not broken, but it is not used as a main claim.

4.3 Experimental Conditions

We vary three factors: access, view, and intervention.

Access. Access conditions separate retrieval failure from post-recall failure. In the retrieved condition, the model receives evidence from a standard retrieval pipeline. In the oracle condition, it receives $\mathcal{A}_q^*$. In the oracle-plus-distractors condition, it receives $\mathcal{A}_q^*$ together with irrelevant, stale, or confounding evidence. The key post-recall analyses are performed under oracle and oracle-plus-distractors conditions.

View. View conditions test whether the recalled evidence is exposed in an evaluable form. The flat view concatenates evidence as natural-language snippets. The source-indexed view exposes item IDs, source IDs, timestamps, statuses, and field names. The relation-explicit view exposes typed edges such as newer-than, overrides, same-event, and contradicts. The state view exposes allowed decision-state variables, including candidate records, typed operands, source priorities, missingness flags, conflict flags, and relation labels.

Oracle-state views are controlled to avoid answer leakage. They may expose operands and state variables required by the gold program, but they may not expose the final answer, the final natural-language conclusion, or the output of the target operator. For lookup queries where the minimal state is identical to the answer, oracle-state views are excluded from the main comparison or reported only as an upper bound.

Intervention. Intervention conditions compare proposal shaping and evaluation-centric computation. The proposal-shaping baselines are direct answering, chain-of-thought, and structured scratchpad. The evaluation-centric interventions are operator harnesses and verifier harnesses. Operator harnesses execute deterministic operations over extracted fields. Verifier harnesses check whether candidate answers satisfy evidence-defined constraints.

4.4 Complexity Controls

We vary evaluation complexity along five axes. Binding count k is the number of variables that must be simultaneously bound. Operator count o is the number of discrete operations in the gold program. Override depth c is the length of stale-to-current update chains. Conflict degree m is the number of mutually inconsistent or priority-ranked evidence items. Missingness level measures how much of the answer-determining state is absent or underdetermined, requiring abstention.

Semantic complexity and context length are naturally coupled: more variables often require more evidence. We therefore use two protocols. In the natural-scaling protocol, evidence size is allowed to grow with complexity. This reflects realistic memory use. In the padded-control protocol, total

context length and distractor count are approximately matched across complexity levels by adding irrelevant or template filler evidence. A complexity effect is considered robust when it appears under both protocols. If it appears only under natural scaling, we report it as a combined complexity-and-context effect.

4.5 Models and Scope of Claims

We evaluate models in tiers. Smaller open-weight models are used for large-scale sweeps, complexity curves, and error taxonomy. Stronger open-weight or mid-scale models are used to test whether the trends survive increased capability. Frontier closed models are used for smaller confirmation runs.

Claims are scoped by model tier. A failure mode that appears only in smaller models is reported as scale-dependent. A failure mode that persists across stronger models under oracle evidence is treated as stronger evidence for a general post-recall evaluation bottleneck.

4.6 Metrics

The primary metric is semantic accuracy:

$$\eta(y) \in \mathcal{Z}_q(\mathcal{A}). \quad (19)$$

We also report semantic invalid rate, unsupported answer rate, binding accuracy, operator accuracy, abstention accuracy, and conflict-resolution accuracy. For view analysis, we report VIR and the correlation between view insufficiency and semantic error. For intervention analysis, we report verifier precision, verifier coverage, rejection rate, abstention rate, tool-call cost, token cost, and latency.

We report two gaps. The oracle-recall gap measures the improvement from retrieved evidence to oracle evidence. The post-recall gap measures the remaining difference between oracle evidence with a flat view and oracle evidence with state views or harnessed evaluation.

4.7 Analysis Plan

The first analysis tests complexity scaling under oracle evidence. We plot semantic accuracy and invalid rate against k , o , c , m , and missingness level. This directly tests whether post-recall failure follows the complexity axes predicted by the dynamic propagation bound.

The second analysis tests view insufficiency. We measure whether errors concentrate in query-view pairs where required fields, relations, or state indicators are absent. We compare flat, source-indexed, relation-explicit, and state views under the same oracle evidence. The key test is whether performance gains track reductions in VIR, rather than merely changing the prompt format.

The third analysis compares prompting-only and harnessed evaluation. We test whether chain-of-thought and scratchpads reduce errors by improving proposal quality, and whether operator or verifier harnesses further reduce semantic invalidity by executing or checking evidence-defined operations.

The final analysis assigns oracle-evidence failures to error categories: view error, binding error, temporal or state error, operator error, conflict error, abstention error, unsupported

Condition	Access	Selection	View	Score
Retrieval SGM	system	system	SGM	–
NIAH SGM	oracle pool	system	SGM	–
Oracle SGM	oracle	oracle	SGM	–
Oracle Raw	oracle	oracle	raw	–
Role View	oracle	oracle	role	–

Table 2: Planned ATM-Bench diagnostic conditions. Scores will be filled after experiments.

generation, and decoding error. The purpose is not to show that hard queries are hard, but to show that post-recall errors are structured and can be predicted from the required evaluation program.

Together, these experiments determine the strength of the contribution. If oracle evidence removes most errors, the bottleneck is mainly access. If errors persist and scale with evaluation complexity, the results support a post-recall evaluation bottleneck. If errors correlate with view insufficiency and decrease under state views, the results support decision-sufficient view as a mechanism. If harnesses reduce semantic invalidity beyond prompting-only methods, the results support evaluation-centric memory reasoning rather than retrieval-only memory improvement.

5 STALE as a State-Validity Case Study

STALE narrows long-term memory to a specific memory-use subtask: determining whether a later observation implicitly invalidates an earlier memory (Chao et al. 2026). This setting is valuable because the failure is not ordinary fact retrieval. The system must revise state, resist stale premises, and adapt actions to the currently authorized state.

5.1 Primitive-Role Alignment

In our terminology, stale-state validity requires a state-oriented artifact substrate and a primitive stack that can maintain and expose current authority. A stylized primitive set is

$$\mathcal{P}_{\text{STALE}} = \{\rho_{\text{extract}}, \rho_{\text{propagate}}, \rho_{\text{adjudicate}}, \rho_{\text{authorize}}, \rho_{\text{readout}}\}. \quad (20)$$

These primitives correspond to extracting state changes, propagating implications through a dependency structure, adjudicating conflicts, authorizing the current state, and reading out the evidence needed for a query.

The decision-facing path is:

$$\mathcal{D}_t \rightarrow \mathcal{D}_t^{\text{auth}} \rightarrow v_u^{\text{state}} \rightarrow \hat{y}. \quad (21)$$

This path makes explicit why retrieval visibility is insufficient. The model must know which state is current, why an older premise is stale, and how this should govern the answer.

5.2 Probe-Level Interpretation

STALE uses three probe families:

- **State Resolution:** whether the system identifies the current state.

System/View	SR	PR	IPA
Retrieval-only readout	–	–	–
Authorized state readout	–	–	–
Role-labeled state view	–	–	–

Table 3: Planned STALE probe-level reporting. SR: State Resolution; PR: Premise Resistance; IPA: Implicit Policy Adaptation.

- **Premise Resistance:** whether the system rejects or corrects a question that embeds a stale premise.
- **Implicit Policy Adaptation:** whether the system adapts recommendations or actions to the updated state.

These probes map naturally onto evidence-view roles: current authority, stale-premise marking, dependency explanation, and action-grounding support.

5.3 Planned Reporting

The STALE section will report whether a specialized state artifact and primitive stack improves the exposure of current-validity roles, and which probe families benefit most. If the experiments show limited gains, this section will be reframed as a limitation analysis rather than a positive method claim.

6 Related Work

6.1 Long-Term Agent Memory

Long-term agent memory systems externalize past interaction into persistent records, summaries, skills, rules, and state. Our work is complementary to this line: it does not replace compression or storage mechanisms, but asks how their outputs are accessed and exposed at runtime.

6.2 Retrieval and Structured Memory

Retrieval-augmented generation and graph-based memory systems provide access methods beyond full-context prompting. Systems such as hierarchical graph memory and direct corpus interaction suggest that access language matters: top- k similarity search, graph browsing, shell-like corpus operations, and workspace interaction expose different information under different budgets. Our framework treats these as different primitive sets rather than as interchangeable retrieval variants (Tang et al. 2026).

6.3 Benchmarks for Memory Use

Personalized memory QA benchmarks test whether agents can answer referential questions grounded in a user’s past. State-invalidation benchmarks test whether agents can revise old beliefs in light of later observations. We use these benchmarks not only for task scores, but as controlled environments for decomposing where memory-use failures occur.

6.4 Evidence, Provenance, and Traceability

Work on evidence grounding, provenance, execution tracing, and failure attribution is closely related to our notion of evidence views. The distinction is that we focus on what structure must be visible to the downstream decision model,

rather than only whether the system can recover a post-hoc explanation.

7 Discussion and Limitations

The framework is intentionally broader than the experiments planned in this paper. Typed artifacts include skills, rules, traces, and workspace files, but the operational layer studied here is evidence-view construction for decision support. We do not claim to solve full skill execution, rule enforcement, or workspace editing.

The empirical scope is also limited. ATM-Bench supports a clean diagnostic separation among retrieval-based, NIAH, Oracle, and Raw conditions, but it does not cover every long-horizon agent workflow. STALE provides a narrow and controlled state-validity setting, but its specialized mechanisms may depend on schema assumptions. The paper should therefore avoid claims that a single access architecture solves long-term memory in general.

The main intended contribution is a decomposition and diagnostic lens. After experiments are complete, this section should state exactly which failure modes were observed, which interventions helped, and which claims remain speculative.

8 Conclusion

Long-horizon agents need more than compressed memories. They need runtime access interfaces that match artifact usage semantics and evidence views that expose the roles required by current decisions. This paper frames that requirement as Evidence View Realization, separating artifact storage, typed access primitives, primitive traces, evidence-view construction, and downstream decoding.

The final version will use ATM-Bench and STALE experiments to determine how far this decomposition explains real memory-use failures and whether role-aware evidence views or state-validity primitive stacks improve downstream decisions.

References

- Alam, S.; Siam, S. I.; Proulx, M. J.; Fort, J.; Newcombe, R.; Kim, H. J.; and Zhang, M. 2026. SuperMemory-VQA: An Egocentric Visual Question-Answering Benchmark for Long-Horizon Memory. *arXiv:2606.00825*.
- Banerjee, P.; Moshtaghi, M.; Subramanian, S.; Misra, A.; and Chadha, A. 2026. APEX-MEM: Agentic Semi-Structured Memory with Temporal Reasoning for Long-Term Conversational AI. *arXiv:2604.14362*.
- Cao, S.; He, J.; and Tan, F. 2026. HiGMem: A Hierarchical and LLM-Guided Memory System for Long-Term Conversational Agents. *arXiv:2604.18349*.
- Chao, H.; Bai, Y.; Sheng, R.; Li, T.; and Sun, Y. 2026. STALE: Can LLM Agents Know When Their Memories Are No Longer Valid? *arXiv:2605.06527*.
- Chhikara, P.; Khant, D.; Aryan, S.; Singh, T.; and Yadav, D. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint arXiv:2504.19413*.
- Deng, X.; Zhong, R.; Peng, H.; Lu, X.; Wu, Y.; Li, G.; Xu, B.; Yao, Y.; Fang, J.; Cao, H.; Guo, J.; Yuan, Y.; Ma, Z.; Yu, Y.; Hu, R.; Dong, B.; Zhu, H.; and Zhang, N. 2026. MemTrace: Tracing and Attributing Errors in Large Language Model Memory Systems. *arXiv:2605.28732*.
- Deng, Y. 2026a. Entity-Collision: A Stratified Protocol for Attributing Retrieval Lift in Agent Memory. *arXiv:2605.29630*.
- Deng, Y. 2026b. Substrate Asymmetry in User-Side Memory: A Diagnostic Framework. *arXiv:2606.11712*.
- Dhayalkar, S. R. 2026. Overcoming the Impedance Mismatch: A Theoretical Roadmap for Fusing Foundation Models and Knowledge Graphs. *arXiv:2606.15656*.
- Fu, J.; Chen, J.; Wang, M.; He, A.; Sheng, M.; Ke, X.; Zhu, Y.; and Gao, Y. 2026. VikingMem: A Memory Base Management System for Stateful LLM-based Applications. *arXiv:2605.29640*.
- Gutiérrez, B. J.; Shu, Y.; Gu, Y.; Yasunaga, M.; and Su, Y. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Gutiérrez, B. J.; Shu, Y.; Qi, W.; Zhou, S.; and Su, Y. 2025. From RAG to Memory: Non-Parametric Continual Learning for Large Language Models. *arXiv:2502.14802*.
- Ji, S.; Li, Y.; and Hooi, B. 2026. Memory is Reconstructed, Not Retrieved: Graph Memory for LLM Agents. *arXiv:2606.06036*.
- Kang, J.; Ji, M.; Zhao, Z.; and Bai, T. 2025. Memory OS of AI Agent. *arXiv:2506.06326*.
- Liu, J.; Su, Y.; Xia, P.; Zhou, Y.; Han, S.; Zheng, Z.; Xie, C.; Ding, M.; and Yao, H. 2026. SimpleMem: Efficient Lifelong Memory for LLM Agents. *arXiv preprint arXiv:2601.02553*.
- Liu, S.; Shang, Y.; and Zhang, X. 2025. TruthfulRAG: Resolving Factual-level Conflicts in Retrieval-Augmented Generation with Knowledge Graphs. *arXiv:2511.10375*.
- Mei, J.; Chen, J.; Yang, G.; Hou, X.; Li, M.; and Byrne, B. 2026. According to Me: Long-Term Personalized Referential Memory QA. *arXiv preprint arXiv:2603.01990*.
- Qiu, J.; Han, Z.; and Huang, C. 2026. SURE-RAG: Sufficiency and Uncertainty-Aware Evidence Verification for Selective Retrieval-Augmented Generation. *arXiv:2605.03534*.
- Rajesh, S.; Holur, P.; Duan, C.; Chong, D.; and Roychowdhury, V. 2026. Beyond Fact Retrieval: Episodic Memory for RAG with Generative Semantic Workspaces. *arXiv:2511.07587*.
- Salemi, A.; Zeng, C.; Nijasure, A.; Chung, J.-H.; Rahimi, R.; Diaz, F.; and Zamani, H. 2026. GrepSeek: Training Search Agents for Direct Corpus Interaction. *arXiv:2605.29307*.
- Tang, Z.; Yu, X.; Xiao, Z.; Wen, Z.; Li, Z.; Zhou, J.; Wang, H.; Wang, H.; Huang, H.; Deng, W.; Sun, F.; and Zhang, Q. 2026. Mnemis: Dual-Route Retrieval on Hierarchical Graphs for Long-Term LLM Memory. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*. San Diego, California, USA: Association for Computational Linguistics. Accepted to ACL 2026 Main Conference.

Tao, X.; Wang, Y.; Cai, Y.; Yang, Z.; and Tang, J. 2026a. Are LLMs Really Not Knowledgeable? Mining the Submerged Knowledge in LLMs' Memory. arXiv:2412.20846.

Tao, Z.; Zhao, J.; Liu, P.; Xi, D.; Chen, Y.; Xu, W.; and Li, Z. 2026b. MemConflict: Evaluating Long-Term Memory Systems Under Memory Conflicts. arXiv:2605.20926.

van Krieken, E.; Acar, E.; and van Harmelen, F. 2020. Analyzing Differentiable Fuzzy Logic Operators. arXiv:2002.06100.

Wang, Y.; Xu, H.; Gu, R.; Ye, Y.; Chen, X.; Mu, X.; Gao, Y.; Guo, C.; Wei, P.; Gu, J.; Li, H.; Chen, K.; and Shou, L. 2026. MedMemoryBench: Benchmarking Agent Memory in Personalized Healthcare. arXiv:2605.11814.

Xu, B.; Chen, Y.; Fang, J.; Zhong, R.; Yao, Y.; Zhu, Y.; Du, L.; and Deng, S. 2026a. StructMem: Structured Memory for Long-Horizon Behavior in LLMs. arXiv:2604.21748.

Xu, W.; Liang, Z.; Mei, K.; Gao, H.; Tan, J.; and Zhang, Y. 2025. A-Mem: Agentic memory for llm agents. In *Advances in Neural Information Processing Systems*.

Xu, Z.; Zhang, S.; Poyraz, E.; Li, Y.; Jin, Y.; Lu, X.; Gu, X.; Ramgopal, K.; Bodigutla, P. K.; and Wang, X. 2026b. Hierarchical Long-Term Semantic Memory for LinkedIn's Hiring Agent. arXiv:2604.26197.

Yue, J.; Hu, C.; Sheng, J.; Zhou, Z.; Zhang, W.; Liu, T.; Guo, L.; and Deng, Y. 2026. HyperMem: Hypergraph Memory for Long-Term Conversations. arXiv:2604.08256.

Zhang, X.; Wang, G.; Cui, Y.; Qiu, W.; Li, Z.; Zhu, B.; and He, P. 2026a. Experience Compression Spectrum: Unifying Memory, Skills, and Rules in LLM Agents. arXiv:2604.15877.

Zhang, Y.; Li, Y.; Payani, A.; and Wang, L. 2026b. AdaMEM: Test-Time Adaptive Memory for Language Agents. In *Proceedings of the 43rd International Conference on Machine Learning*, Proceedings of Machine Learning Research.

Zhou, C.; Chai, H.; Chen, W.; Guo, Z.; Shan, R.; Song, Y.; Xu, T.; Yang, Y.; Yu, A.; Zhang, W.; Zheng, C.; Zhu, J.; Zheng, Z.; Zhang, Z.; Lou, X.; Zhang, C.; Fu, Z.; Wang, J.; Liu, W.; Lin, J.; and Zhang, W. 2026. Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering. arXiv:2604.08224.

Appendix

The appendix is organized to keep reproducibility and diagnostic detail outside the main narrative:

1. Appendix A: formal details and notation.
2. Appendix B: ATM-Bench data, prompts, evidence views, and evaluation details.
3. Appendix C: STALE data, primitive-role mapping, and evaluation details.
4. Appendix D: qualitative error taxonomy.
5. Appendix E: additional tables and robustness checks.

A Formal Details

This appendix gives the full proofs for the two formal claims in Section 3.

Proof of View Collapse Impossibility

Let D be any decoder whose input is only the view $V_q(\mathcal{A})$. Suppose

$$V_q(\mathcal{A}_1) = V_q(\mathcal{A}_2)$$

and

$$\mathcal{Z}_q(\mathcal{A}_1) \cap \mathcal{Z}_q(\mathcal{A}_2) = \emptyset.$$

For a deterministic decoder, the identical view implies that the decoder outputs the same response y in both evidence worlds. If y is evidence-valid for $\mathcal{A}_1$, then

$$\eta(y) \in \mathcal{Z}_q(\mathcal{A}_1).$$

If the same y is evidence-valid for $\mathcal{A}_2$, then

$$\eta(y) \in \mathcal{Z}_q(\mathcal{A}_2).$$

Thus $\eta(y) \in \mathcal{Z}_q(\mathcal{A}_1) \cap \mathcal{Z}_q(\mathcal{A}_2)$, contradicting the disjointness assumption. Therefore, no deterministic view-only decoder can be evidence-valid on both worlds.

For a randomized decoder, the same view induces the same output distribution $p(y \mid V_q)$ in both worlds. To be evidence-valid with probability one on $\mathcal{A}_1$, its support must be contained in

$$\{y : \eta(y) \in \mathcal{Z}_q(\mathcal{A}_1)\}.$$

To be evidence-valid with probability one on $\mathcal{A}_2$, the same support must be contained in

$$\{y : \eta(y) \in \mathcal{Z}_q(\mathcal{A}_2)\}.$$

Because η is single-valued where defined and the semantic answer sets are disjoint, these two valid-output sets are disjoint. No nonempty probability distribution can have support contained in both. Hence a randomized view-only decoder also cannot be evidence-valid on both worlds with probability one.

Proof of Dynamic Propagation of Binding Error

We first bound the one-step binding perturbation. Since all values in S_t equal v_t^* ,

$$\tilde{v}_t = \sum_{i \in S_t} \alpha_{ti} v_i + \sum_{i \notin S_t} \alpha_{ti} v_i = B_t v_t^* + \sum_{i \notin S_t} \alpha_{ti} v_i.$$

Using $\sum_i \alpha_{ti} = 1$, we have

$$\tilde{v}_t - v_t^* = \sum_{i \notin S_t} \alpha_{ti} (v_i - v_t^*).$$

Taking norms and applying the triangle inequality gives

$$\|\tilde{v}_t - v_t^*\|_2 \leq \sum_{i \notin S_t} \alpha_{ti} \|v_i - v_t^*\|_2.$$

By the definition of D_t , $\|v_i - v_t^*\|_2 \leq D_t$ for all i . Hence

$$\|\tilde{v}_t - v_t^*\|_2 \leq \sum_{i \notin S_t} \alpha_{ti} D_t = (1 - B_t) D_t.$$

Now consider the evaluation-state error:

$$e_{t+1} = \|h_{t+1} - h_{t+1}^*\|_2 = \|T_t(h_t, \tilde{v}_t) - T_t(h_t^*, v_t^*)\|_2.$$

By the local Lipschitz condition,

$$e_{t+1} \leq \rho_t \|h_t - h_t^*\|_2 + \beta_t \|\tilde{v}_t - v_t^*\|_2.$$

Substituting $e_t = \|h_t - h_t^*\|_2$ and the binding perturbation bound yields

$$e_{t+1} \leq \rho_t e_t + \beta_t (1 - B_t) D_t.$$

It remains to unroll the recurrence. Repeated substitution gives

$$e_T \leq \left(\prod_{t=0}^{T-1} \rho_t \right) e_0 + \sum_{s=0}^{T-1} \left(\prod_{u=s+1}^{T-1} \rho_u \right) \beta_s (1 - B_s) D_s,$$

which proves the claim.

B ATM-Bench Details

This appendix will document the ATM-Bench splits, memory processors, answer models, judge models, evidence conditions, prompts, and scoring scripts used in the final experiments.

C STALE Details

This appendix will document the STALE scenario types, probe dimensions, state-validity roles, primitive-stack implementation, and scoring protocol.

D Error Analysis

This appendix will contain qualitative examples grouped by substrate failure, access failure, selection failure, view-construction failure, and decoder failure.

E Additional Results

This appendix will contain additional tables, robustness checks, judge agreement analyses, and ablations.